

Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Santa Fe



Análisis y diseño de algoritmos avanzados

Grupo 601

Nombre del profesor:

Octavio Navarro Hinojosa

MA. Actividad: Roomba

Alumno:

Diego Flores Becerril

| A01769546

19 nov 2025

Índice

Introducción.....	3
Desarrollo.....	3
Diseño y Características de los Agentes.....	3
Arquitectura de Subsunción del Agente.....	5
Diseño y Características del Ambiente.....	7
PEAS.....	8
Análisis de resultados.....	9
Simulación 1.....	9
Simulación 2.....	13
Conclusiones.....	15
Anexos.....	16

Introducción

El presente reporte tiene como objetivo desarrollar y analizar una simulación de un sistema de agentes autónomos (Roomba) dentro de un entorno controlado. La problemática consiste en limpiar un espacio de dimensiones $M \times N$ con una distribución aleatoria de suciedad y obstáculos, cuya dificultad consiste en la optimización del tiempo de limpieza y la gestión de energía de los agentes.

La solución propuesta implementa agentes reactivos basados en la arquitectura de subsunción simulada mediante una Máquina de Estados. A diferencia de un recorrido aleatorio simple, los agentes incorporan memoria a corto plazo, mapas de calor de celdas visitadas y estrategias de exploración de largo alcance para maximizar la cobertura del área y minimizar el consumo de energía.

Desarrollo

Diseño y Características de los Agentes

El agente diseñado, RandomAgent, es un agente reactivo con estado interno, ya que aunque toma decisiones paso a paso basadas en su percepción inmediata, mantiene una memoria de las celdas visitadas y una lista de estaciones de carga conocidas para optimizar su comportamiento.

Control Parcial: El agente opera bajo condiciones de control parcial debido a que sus acciones no garantizan un resultado determinista, estando sujetas a la dinámica cambiante de un entorno multiagente. Esta limitación se manifiesta en tres aspectos críticos:

- **Incertidumbre en la Navegación:** Aunque el agente calcule una ruta hacia un objetivo (como una estación de carga), no tiene control sobre los otros agentes. Un camino que inicialmente parecía libre puede ser obstruido repentinamente por otro agente, obligando al robot a recalcular su ruta y gastar energía no prevista.
- **Fallo por Agotamiento de Recursos:** El agente puede perder totalmente el control si su batería se agota. Dado que el entorno es dinámico, existe el riesgo de que desviaciones forzadas o tiempos de espera en estaciones ocupadas consuman la energía de reserva antes de lograr recargar, llevando a la "muerte" del agente.
- **Redundancia de Acciones:** Debido a que el agente sólo percibe su vecindad inmediata, puede decidir moverse a limpiar una celda que, en el mismo instante de tiempo, está siendo limpiada por otro agente, resultando en una acción ineficaz.

Capacidad efectora: El agente es capaz de realizar cambios en el entorno y también para modificar su propio estado:

- Limpiar: a través del método `clean()`, el agente cambia el estado de una celda con suciedad (FloorAgent) de “sucio” a “limpio”, cumpliendo el objetivo principal de la simulación
- Movimiento y Carga: Mediante `move_agent()` y `charge()`, el agente altera su posición en el grid y su nivel de batería interna.
- Comunicación: A través de `share_station_location()`, el agente tiene la capacidad de compartir (modificar) el conocimiento de otros agentes (de su mismo tipo [RandomAgent]) al compartir la ubicación de las estaciones de carga descubiertas

Proactividad: El agente es capaz de mostrar un comportamiento dirigido a metas tomando la iniciativa para satisfacer objetivos:

- El agente no se limita a reaccionar a la suciedad inmediata (comportamiento puramente reactivo); en su lugar, monitorea su eficiencia de exploración a través de un mapa de visitas interno. Cuando detecta un estancamiento en una zona limpia (mínimo local), el agente toma la iniciativa de fijar un objetivo de navegación lejano (STATE_TRAVELING), desplazándose activamente hacia áreas inexploradas para maximizar su métrica de desempeño de limpieza total, anticipándose a la falta de suciedad en su vecindad actual.

Reactividad: La toma de decisiones se implementa como un mapeo directo de la situación percibida a una acción inmediata. La arquitectura de subsunción de RandomAgent (las prioridades en `determine_next_state`) es un conjunto de reglas de reacción ante estímulos específicos.

- Estímulo (Percepción): Detectar una celda sucia (FloorAgent no limpio).
- Reacción (Acción): Cambiar inmediatamente al estado CLEANING para limpiar.
- Estímulo (Percepción): Detectar un obstáculo o pared.
- Reacción (Acción): Evitar esa celda y elegir otra dirección libre.

Habilidad Social: Esta característica se implementa a través de dos mecanismos de interacción directa e indirecta:

- Comunicación Cooperativa: Cuando un agente detecta una estación de carga (StationAgent) mediante sus sensores locales, comparte inmediatamente las coordenadas de este recurso con el resto de los agentes en la simulación que están en su vecindario mediante el método `share_station_location()`. Esta interacción social permite que agentes que se encuentran en zonas

críticas de batería conozcan la ubicación de cargadores que no han explorado personalmente, aumentando la tasa de supervivencia del grupo.

- **Coordinación en el uso de Recursos:** Dado que las estaciones de carga son un recurso limitado y compartido, los agentes implementan una conducta social de "espera". Antes de moverse a una estación, verifican su disponibilidad mediante `is_station_occupied()`. Si la estación está siendo utilizada por otro agente, el solicitante inhibe su movimiento y espera su turno, evitando conflictos físicos y gestionando eficientemente el acceso a la energía.

Racionalidad: La racionalidad del RandomAgent no se basa en la omnisciencia (conocer la ubicación de toda la suciedad), sino en la optimización de decisiones bajo incertidumbre para maximizar las métricas definidas en el modelo. Además, el agente recopila información a través de sus sensores y puede actuar por su propia cuenta acorde a la arquitectura de subsunción y la misma prioridad dentro de los métodos definidos.

La racionalidad del sistema se evalúa cuantitativamente en la clase RandomModel a través de tres métricas de desempeño clave recopiladas por el DataCollector:

- **Porcentaje de Limpieza (`clean_percentage`):** Es la medida de éxito principal. El agente demuestra racionalidad al priorizar el estado CLEANING sobre cualquier otro cuando percibe suciedad, garantizando la maximización directa de esta métrica.
- **Eficiencia Energética (`average_energy`):** El agente incorpora conocimiento previo sobre su consumo energético (1% por movimiento). Su comportamiento racional se manifiesta al interrumpir la tarea principal para buscar una estación de carga cuando su energía es crítica (< 25%), sacrificando tiempo a corto plazo para asegurar la operatividad a largo plazo.
- **Eficiencia de Movimiento (`average_movements`):** En lugar de moverse aleatoriamente (lo cual sería irracional dado que desperdicia energía sin garantizar cobertura), el agente selecciona el movimiento que estadísticamente le ofrece la mayor probabilidad de encontrar nuevas celdas sucias, optimizando así la relación entre energía gastada y área cubierta.

Arquitectura de Subsunción del Agente

El RandomAgent implementa una arquitectura de subsunción dividida en 5 capas de comportamiento. Cada capa funciona como un módulo de decisión independiente que compite por el control de los

actuadores del agente. Las capas inferiores poseen mayor prioridad y, cuando sus condiciones de disparo se cumplen, interrumpen la ejecución de las capas superiores.

A continuación se detalla la jerarquía de abajo hacia arriba (de mayor a menor prioridad):

Capa 0: Supervivencia Crítica (Estado: CHARGING)

- Prioridad: Máxima.
- Estímulo (Sensor): Detección de conexión a estación (is_on_station) Y batería incompleta ($\text{energy} < 100$).
- Comportamiento: El agente permanece inmóvil recargando energía.
- Subsunción: Inhibe cualquier intento de movimiento, exploración o limpieza. Es una restricción física fundamental; el agente no puede operar si está "enchufado" recuperando energía vital.

Capa 1: Autonomía / Supervivencia Preventiva (Estado: SEEKING_CHARGER)

- Prioridad: Alta.
- Estímulo (Sensor): Nivel de energía por debajo del umbral de seguridad ($\text{energy} < 25$).
- Comportamiento: Navegación directa hacia la estación de carga óptima.
- Subsunción: Suprime el instinto de trabajo (CLEANING). Aunque el agente perciba suciedad en su camino, la ignora para priorizar su supervivencia, evitando quedar inoperativo lejos de la base.

Capa 2: Trabajo / Oportunismo (Estado: CLEANING)

- Prioridad: Media.
- Estímulo (Sensor): Detección de suciedad en la celda actual (FloorAgent sucio).
- Comportamiento: Ejecutar acción de limpieza.
- Subsunción: Interrumpe momentáneamente los comportamientos de navegación (TRAVELING o WANDERING). Si el agente estaba viajando a un destino lejano pero "tropieza" con suciedad, la arquitectura le obliga a detenerse y limpiar (comportamiento oportunista).

Capa 3: Navegación Global (Estado: TRAVELING)

- Prioridad: Baja.
- Estímulo (Memoria): Existencia de un objetivo de exploración lejano ($\text{exploration_target}$ is not None), activado por la detección de mínimos locales (aburrimiento).

- Comportamiento: Movimiento dirigido hacia coordenadas distantes no visitadas.
- Subsunción: Inhibe la exploración local aleatoria. Evita que el agente pierda tiempo revisitando celdas cercanas ya limpias, forzándolo a desplazarse a nuevas áreas.

Capa 4: Exploración Local (Estado: WANDERING)

- Prioridad: Mínima (Comportamiento por defecto).
- Estímulo: Ausencia de estímulos de capas superiores.
- Comportamiento: Movimiento basado en la heurística de Vecino Menos Visitado (LVN) con inercia (anti-backtracking).
- Función: Garantiza que el agente siempre tenga una acción que realizar, cubriendo sistemáticamente el área inmediata cuando no hay emergencias ni suciedad visible.

Diseño y Características del Ambiente

Inaccesible: El agente no puede obtener información completa, precisa y actualizada sobre el estado global del mismo. Los agentes no cuentan con una visión omnisciente del cuarto; su percepción está limitada exclusivamente a su ubicación actual y a sus celdas vecinas inmediatas. Debido a esta restricción, el agente desconoce inicialmente la ubicación de la suciedad, los obstáculos y las estaciones de carga que se encuentran fuera de su rango visual, obligándolo a construir un mapa interno (visited_cells) mediante la exploración para compensar esta falta de información.

No Determinista: No se puede garantizar un único efecto resultante al realizar una acción. Aunque las reglas de movimiento del agente están programadas, existe incertidumbre sobre el estado final debido a la interacción concurrente con otros agentes. Por ejemplo, un agente puede decidir moverse a una estación de carga que percibe como libre, pero al llegar, esta podría haber sido ocupada por otro agente en el mismo instante de tiempo.

No Episódico: El desempeño del agente no depende de episodios aislados, sino que las decisiones actuales afectan a las futuras. En este caso, el agente debe razonar sobre las consecuencias a largo plazo de sus acciones, especialmente en la gestión de energía. La decisión de realizar un viaje largo de exploración (Traveling) en el paso actual consume batería que será necesaria en pasos futuros para regresar a la estación de carga.

Dinámico: Operan otros procesos (los otros agentes) que cambian el entorno de maneras que salen del control del agente individual. Mientras un agente elige su siguiente movimiento, el estado del cuarto cambia: las celdas que el agente tenía planeado limpiar pueden ser limpiadas por otros compañeros, y las estaciones de carga disponibles pueden saturarse.

Discreto: Existe un número fijo y finito de percepciones, acciones y estados. La simulación se lleva a cabo dentro de un grid de dimensiones predeterminadas ($M \times N$), donde el tiempo avanza en pasos discretos (steps). Las posiciones de los agentes están limitadas a coordenadas enteras, y los estados de las celdas son finitos (limpio, sucio, ocupado, obstáculo), lo que delimita claramente las combinaciones posibles dentro del sistema.

PEAS

Performance:

- Limpieza: Maximizar el porcentaje de celdas limpias (clean_percentage). El objetivo principal es llevar este valor al 100% antes de que termine el tiempo.
- Supervivencia: Mantener el nivel de energía por encima de 0. Un agente muerto penaliza el desempeño del enjambre.
- Eficiencia: Minimizar la cantidad total de movimientos (average_movements) necesarios para limpiar todo el cuarto, evitando recorridos redundantes sobre áreas ya limpias.

Environment:

- Físico: Un grid discreto de dimensiones ($M \times N$) delimitado por bordes.
- Contenido: El entorno contiene obstáculos fijos (ObstacleAgent), suciedad estática pero removible (FloorAgent), estaciones de carga fijas (StationAgent) y agentes móviles dinámicos (RandomAgent).
- Dinámica: El entorno cambia por la acción de los agentes (la suciedad desaparece, las estaciones se ocupan) y presenta incertidumbre por la concurrencia de movimientos.

Actuators:

- Sistema de Movimiento (move_agent): Mecanismo para desplazarse a una de las 8 celdas adyacentes libres, consumiendo 1% de batería por acción.
- Mecanismo de Limpieza (clean): Capacidad para alterar el estado de la celda actual, eliminando al agente de suciedad (FloorAgent) y consumiendo energía.

- Sistema de Carga (charge): Capacidad de acoplarse a una estación para recuperar 5% de batería por turno.
- Mecanismo de Espera: Capacidad de detener los motores voluntariamente para esperar turno en una estación ocupada sin consumir energía.

Sensors:

- Percepción Local (get_empty_neighbors): Percibe el contenido de la celda actual y sus 8 vecinas inmediatas (Obstáculos, Suciedad, Estaciones, Otros Agentes).
- Monitor Interno (self.energy): Autopercepción del nivel de batería para activar estados de supervivencia.
- Sensor de Ocupación Remota (is_station_occupied): Capacidad de detectar si una estación específica dentro de su memoria está siendo utilizada por otro agente.

Análisis de resultados

Simulación 1

- Toma como base las consideraciones generales (anexo 1).
- El agente inicia en la celda [1,1]. En esa posición se encuentra una estación de carga.

Parámetros:

- | | |
|-----------------------|--------------------------------|
| ● Random seed: 42 | ● Number of obstacles: 15 |
| ● Number of agents: 1 | ● Number of dirty tiles: 20 |
| ● Grid width: 20 | ● Maximum number of steps: 100 |
| ● Grid height: 20 | |

Como se observa en la Figura 1, la simulación inicia con el Agente y la Estación de Carga ubicados en la coordenada [1,1]. Inicialmente, el agente opera bajo el estado WANDERING, dado que las condiciones para activar estados de mayor prioridad (como carga crítica o detección de suciedad inmediata) no se han cumplido.

Durante el recorrido, el agente explora sistemáticamente desde el cuadrante inferior izquierdo hacia el derecho, utilizando la heurística de Vecino Menos Visitado (Least Visited Neighbor). Este comportamiento se complementa con un mecanismo de detección de estancamiento ("aburrimiento"),

el cual redirige al agente hacia áreas nuevas cuando detecta que su vecindad actual ha sido visitada excesivamente.

Hacia la fase final (Figura 2), el agente logra alcanzar la zona superior derecha; sin embargo, al descender su energía al umbral crítico ($<25\%$), se activa el comportamiento de supervivencia SEEKING_CHARGER. Utilizando la distancia Manhattan, el agente calcula y ejecuta la ruta de retorno a la estación de carga almacenada en su memoria. Finalmente, la Figura 3 evidencia que la restricción temporal de 100 pasos fue insuficiente para completar la limpieza total del entorno en este escenario específico.

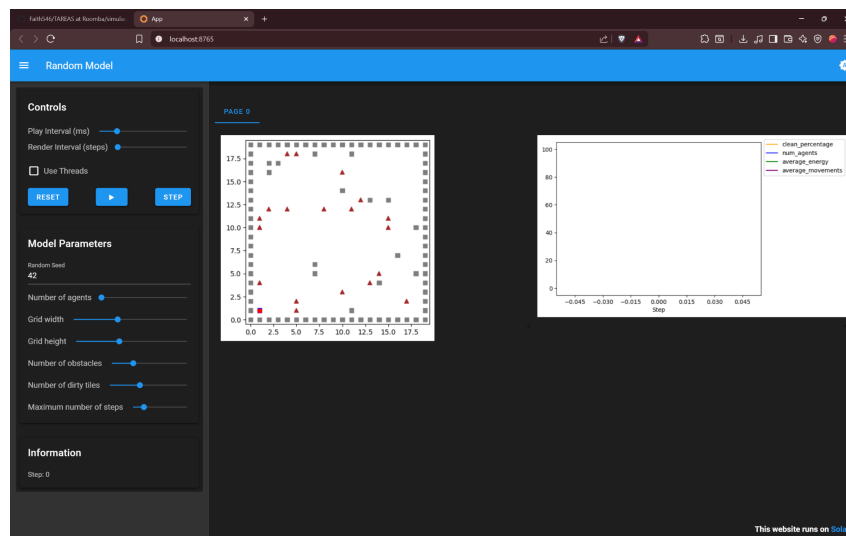


Figura 1. Condiciones iniciales de la simulación 1

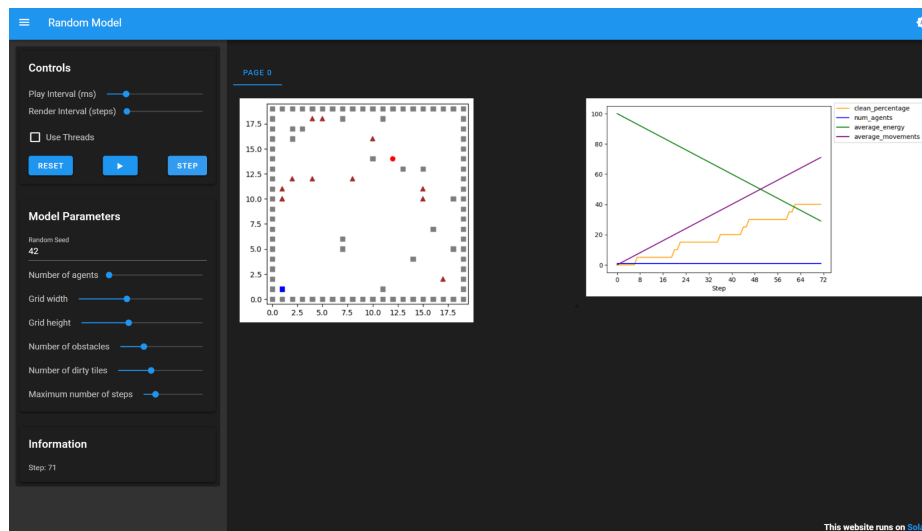


Figura 2. Retorno a la estación de recarga

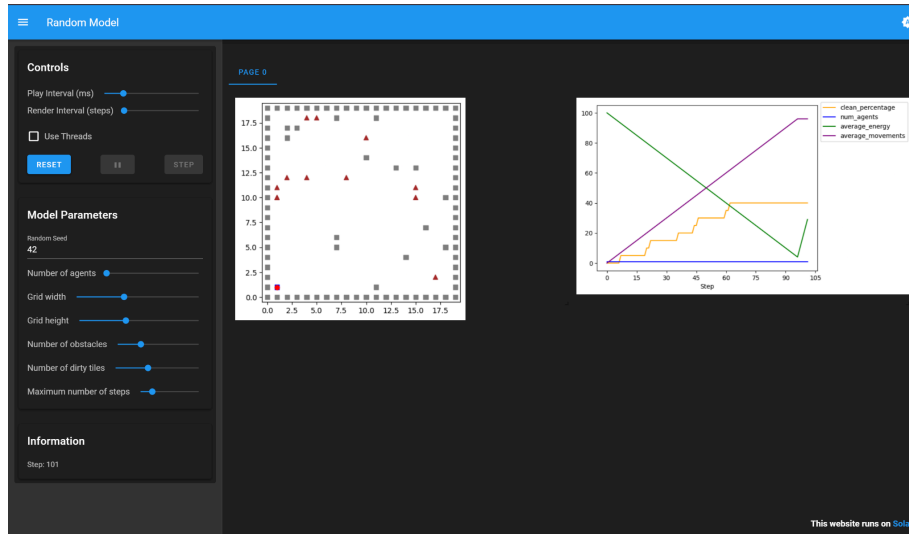


Figura 3. Finalización de la simulación 1 con 100 pasos

En la Figura 4, se observa que el agente acudió a la estación de carga una única vez al alcanzar el umbral del 25% de energía (paso 71). Hasta ese punto, había logrado limpiar el 40% de las celdas, requiriendo un número considerable de movimientos (entre 90 y 100).

Inicialmente, se podría inferir que extendiendo el tiempo de ejecución se lograría la limpieza total en el doble o triple de pasos. Sin embargo, esta hipótesis se descarta al analizar la simulación con un límite de 500 pasos (Figura 5): el agente requirió 414 pasos, más de 250 movimientos efectivos y 4 ciclos de recarga para completar la tarea.

Este aumento en el tiempo se explica mediante la trayectoria observada en la Figura 6. El agente presentó dificultades para localizar la suciedad remanente en la zona inferior derecha. Debido a su comportamiento reactivo y oportunista, al explorar dicha área, el agente desviaba su trayectoria hacia la parte superior cada vez que detectaba otras celdas sucias intermedias, retrasando significativamente la cobertura de la esquina más alejada.

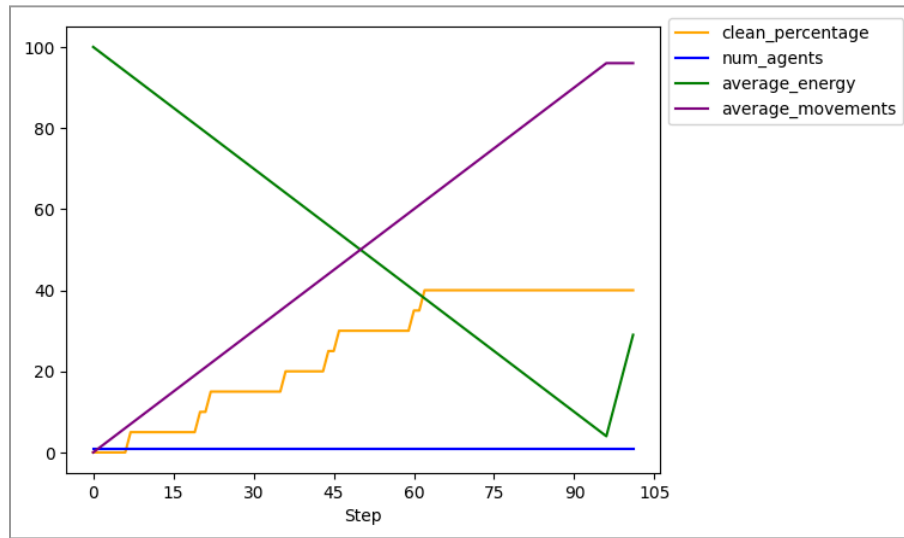


Figura 4. Metricas durante la simulación 1 con 100 pasos

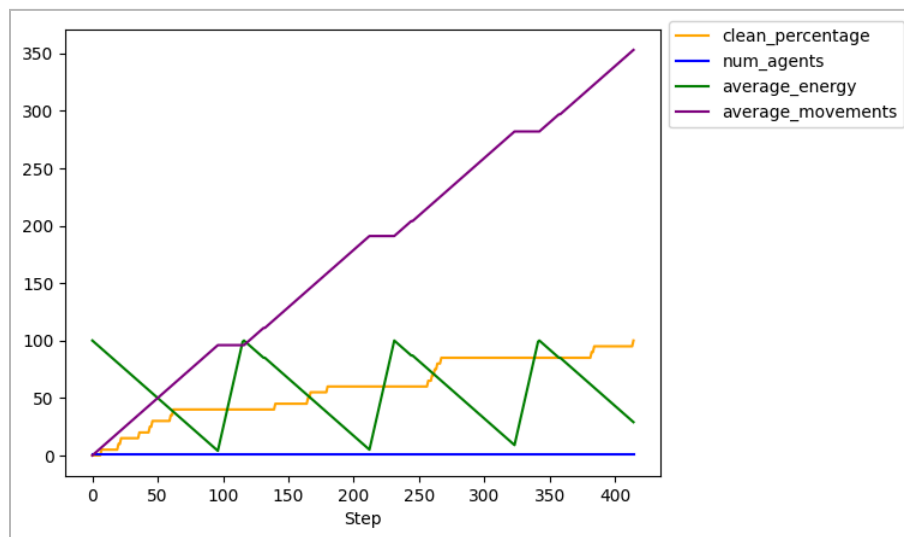


Figura 5. Metricas de la simulación 1 con limite de 500 pasos

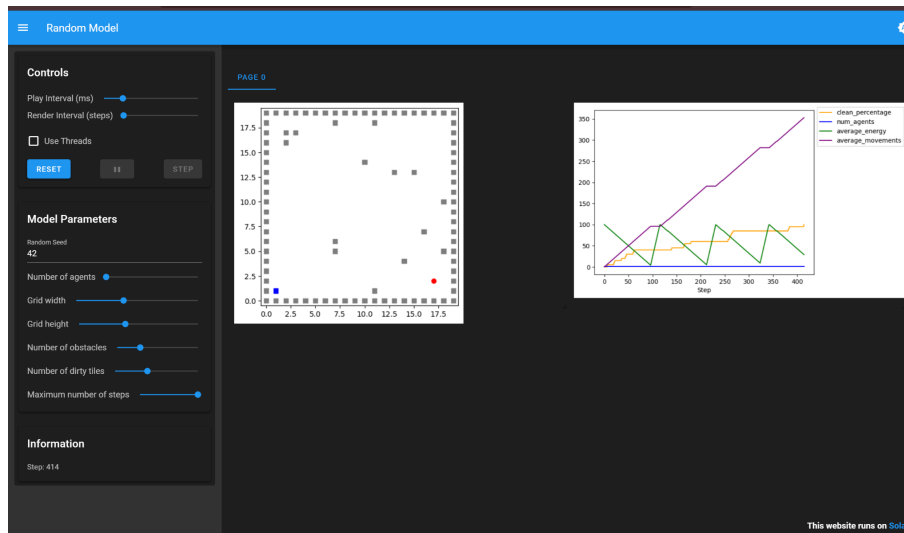


Figura 6. Resultado de la simulación 1 con límite de 500 pasos

Simulación 2

- Toma como base las consideraciones generales.
- Los agentes empiezan en posiciones aleatorias. En esas posiciones van a existir estaciones de carga.
- Los agentes conocen solamente la posición de su estación de carga inicial, pero pueden cargarse en cualquier estación de carga.
- Deberás de recopilar los datos solicitados por cada agente.

Parámetros:

- Random seed: 40
- Number of agents: 10
- Grid width: 20
- Grid height: 20
- Number of obstacles: 15
- Number of dirty tiles: 20
- Maximum number of steps: 100

Debido a la naturaleza de la semilla inicial, la posición de partida de los agentes puede favorecer o dificultar la rapidez de la limpieza, sumado a la distribución aleatoria de obstáculos y suciedad. En la Figura 7 se observa una distribución desigual: existe una concentración de 3 agentes en la parte inferior izquierda, mientras que la zona superior presenta una densidad baja, con apenas 2 agentes en extremos opuestos.

Para la Figura 8, se ha limpiado más del 60% de la suciedad total; sin embargo, persisten celdas sucias en la zona superior derecha. Aunque había un agente cercano, el mecanismo de desempate aleatorio (utilizado cuando las celdas tienen el mismo número de visitas) postergó la exploración inmediata de esa área. Finalmente, la limpieza total se completó en el paso 53, siendo la celda del extremo superior derecho la última en ser procesada (Figura 9).

Como muestra la Figura 10, los 10 agentes mantuvieron una energía promedio del 70% y realizaron cerca de 45 movimientos cada uno, sin registrarse muertes por falta de batería. Esta mejora respecto a la Simulación 1 se atribuye directamente a la cantidad de agentes: al iniciar en posiciones distribuidas, logran una cobertura del área, reduciendo drásticamente el tiempo total y el desgaste individual.

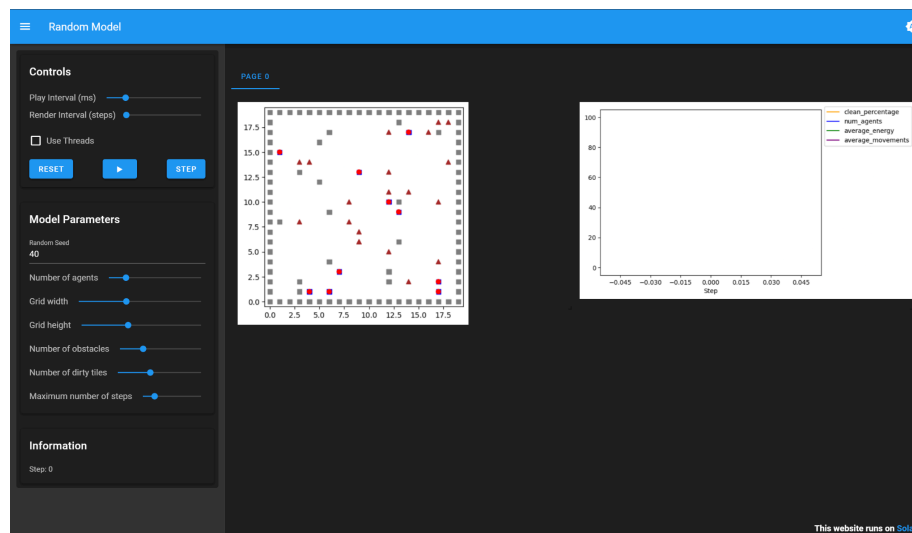


Figura 7. Condiciones iniciales de la simulación 2

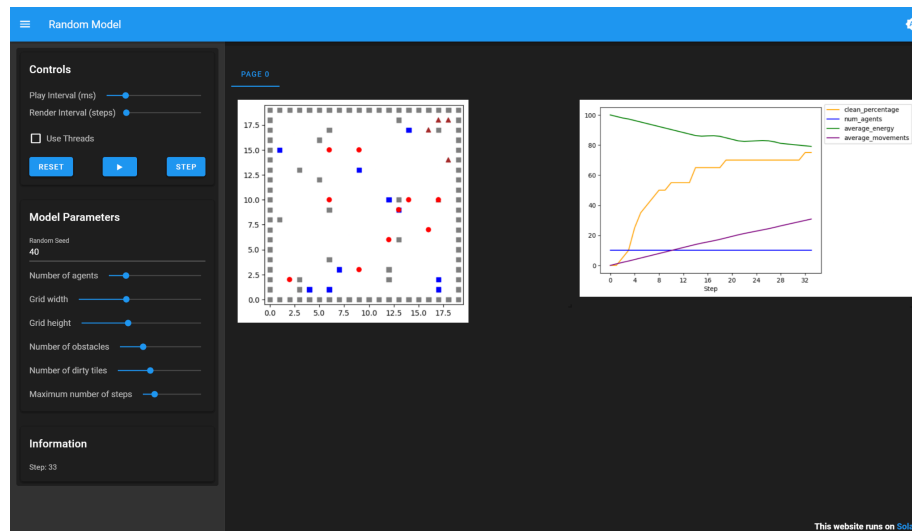


Figura 8. Progreso de limpieza de celdas sucias durante la simulación 2

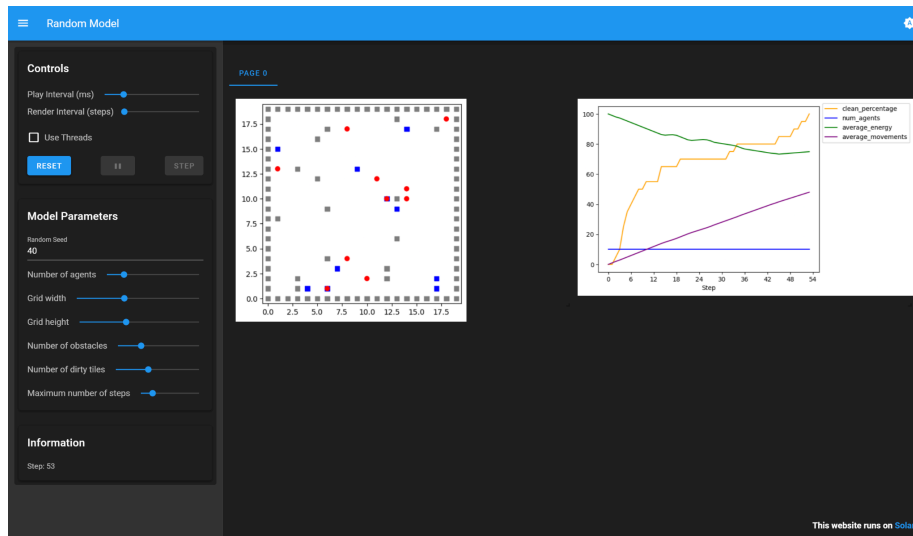


Figura 9. Resultado de la simulación 2 con limite de 100 pasos

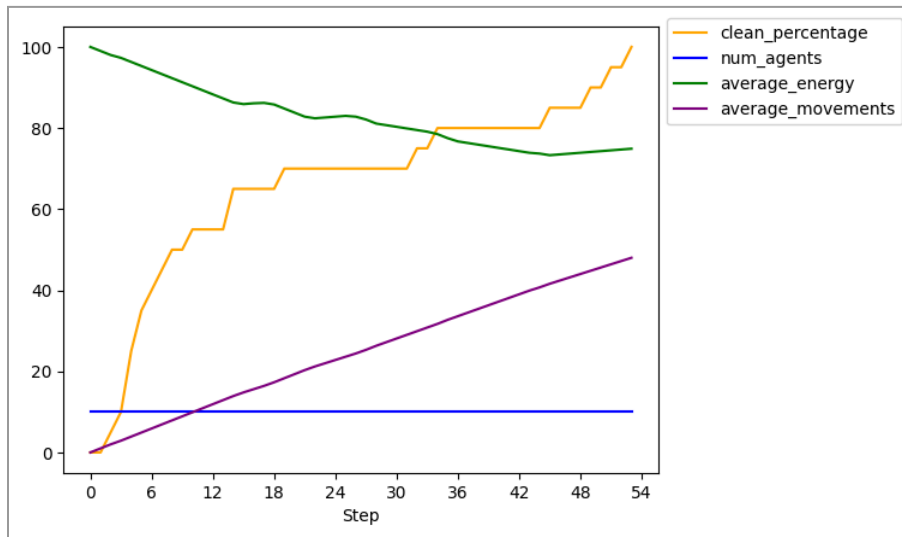


Figura 10. Metricas de la simulación 2 con limite de 100 pasos

Conclusiones

Los resultados obtenidos evidencian un contraste significativo entre los escenarios simulados. Se demostró que el factor determinante en el desempeño del sistema no es solo la capacidad individual, sino la escalabilidad (cantidad de agentes). Al incrementar el número de RandomAgents, se logra una cobertura espacial superior en menor tiempo. Se observó un fenómeno de cooperación implícita: aunque los agentes no trabajan con una coordinación explícita o un plan centralizado, la suma de sus

comportamientos individuales dirigidos a metas (limpiar y sobrevivir) resulta en un comportamiento emergente que satisface eficientemente los objetivos globales del sistema.

El desarrollo de esta actividad permitió dimensionar la complejidad inherente al diseño de agentes inteligentes. La creación de sistemas que actúen eficazmente en entornos dinámicos y parcialmente observables requiere un balance paradójico entre creatividad y lógica rigurosa. No basta con definir reglas simples; es necesario diseñar arquitecturas robustas (como máquinas de estados) que rijan transiciones, decisiones y comportamientos adaptables. Finalmente, se concluye que la robustez de un agente no es accidental, sino el resultado de una preparación deliberada en su diseño para afrontar la incertidumbre y los diversos factores —incluyendo la competencia con otros agentes— que afectan su entorno.

Anexos

1. Consideraciones generales:

- Inicializa las celdas sucias en ubicaciones aleatorias.
- Inicializa las celdas con obstáculos en ubicaciones aleatorias.
- El agente tiene una batería que usa cada que quiera hacer una tarea:
- La batería inicia en 100% de carga.
- Cada acción que realiza el agente le quita 1% de batería. Si no realiza una acción, no pierde batería.
- El agente tiene que tener la capacidad de regresar a cargar batería para poder seguir limpiando.
- Cada episodio que el agente esté en la estación de carga, se recarga 5% de la batería.
- El agente tiene como objetivo limpiar el cuarto lo más que pueda en el tiempo máximo de ejecución, sin quedarse sin batería.
- Deberás recopilar la siguiente información durante la ejecución:
 - Tiempo necesario hasta que todas las celdas estén limpias (o se haya llegado al tiempo máximo).
 - Porcentaje de celdas limpias después del termino de la simulación.
 - Número de movimientos realizados por el agente.

2. [Repositorio de GitHub](#)